

TUGAS BESAR IF3170
TEORI BAHASA FORMAL DAN OTOMATA
MILESTONE 1 - LEXICAL ANALYSIS



Muhammad Ra'if Alkautsar	13523011
Fajar Kurniawan	13523076
Darrel Adinarya Sunanda	13523061
Reza Ahmad Syarif	13523119

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
JL. GANESHA 10, BANDUNG 40132
2025

Daftar Isi

Daftar Isi.....	2
1. Landasan Teori.....	3
a. Token.....	3
b. Lexer (Penganalisis Leksikal).....	3
c. Deterministic Finite Automata.....	3
2. Perancangan & Implementasi.....	5
a. Diagram Deterministic Finite Automata (DFA).....	5
b. Implementasi Tambahan.....	10
3. Pengujian.....	11
4. Kesimpulan & Saran.....	39
a. Kesimpulan.....	39
b. Saran.....	40
Lampiran.....	41

1. Landasan Teori

Dalam pemrosesan bahasa pemrograman, terdapat beberapa tahap yang harus dilalui untuk mengubah kode sumber menjadi program yang dapat dieksekusi. Tahap-tahap ini secara berurutan adalah analisis leksikal (lexical analysis), analisis sintaksis (syntax analysis), analisis semantik (semantic analysis), dan tahap-tahap selanjutnya seperti optimasi kode dan generasi kode. Analisis leksikal akan menghasilkan token-token yang digunakan dalam tahap-tahap selanjutnya.

a. Token

Token sendiri adalah unit atomik terkecil dalam bahasa pemrograman yang memiliki makna. Token merepresentasikan elemen-elemen dasar dalam kode sumber seperti kata kunci, pengidentifikasi (identifier), operator, literal, dan simbol-simbol khusus. Setiap token memiliki dua komponen utama:

- Tipe (Type), yaitu kategori atau jenis token yang membantu kompiler menentukan aturan gramatikal yang berlaku.
- Nilai (Value), yaitu representasi string dari token tersebut. Misalnya, token KEYWORD dengan nilai "program" atau token NUMBER dengan nilai "123.45".

Token memungkinkan kompiler untuk bekerja dengan entitas yang lebih bermakna daripada karakter individual, memudahkan proses analisis sintaksis dan semantik selanjutnya.

b. Lexer (Penganalisis Leksikal)

Lexer atau scanner adalah komponen yang melakukan analisis leksikal, yaitu proses mengubah sekuens karakter dari kode sumber menjadi sekuens token. Lexer adalah fase pertama dari proses kompilasi dan perantara antara teks murni dengan struktur yang lebih bermakna nantinya. Fungsi utama lexer meliputi:

- Membaca karakter input dari kode sumber
- Mengidentifikasi pola karakter yang membentuk token
- Menghilangkan elemen-elemen yang tidak bermakna seperti whitespace dan komentar
- Menghasilkan daftar token untuk diproses oleh parser

Dalam implementasinya, lexer memanfaatkan konsep Deterministic Finite Automata.

c. Deterministic Finite Automata

Deterministic Finite Automaton (DFA) adalah model komputasi teoritis yang mampu menerima alphabet dan berubah state. Sebuah DFA adalah mesin dengan keadaan atau state terbatas yang menerima atau menolak string berdasarkan aturan transisi deterministik, artinya, untuk setiap keadaan dan input terdapat tepat satu transisi yang mungkin. Secara formal, DFA didefinisikan sebagai 5-tuple $(Q, \Sigma, \delta, q_0, F)$ dengan:

- Q : Himpunan berhingga dari keadaan (states)

- Σ : Alfabet input (himpunan simbol yang dikenali)
- δ : Fungsi transisi ($\delta: Q \times \Sigma \rightarrow Q$) yang memetakan pasangan keadaan dan simbol input ke keadaan berikutnya
- q_0 : Keadaan awal ($q_0 \in Q$)
- F : Himpunan keadaan akhir/penerimaan ($F \subseteq Q$)

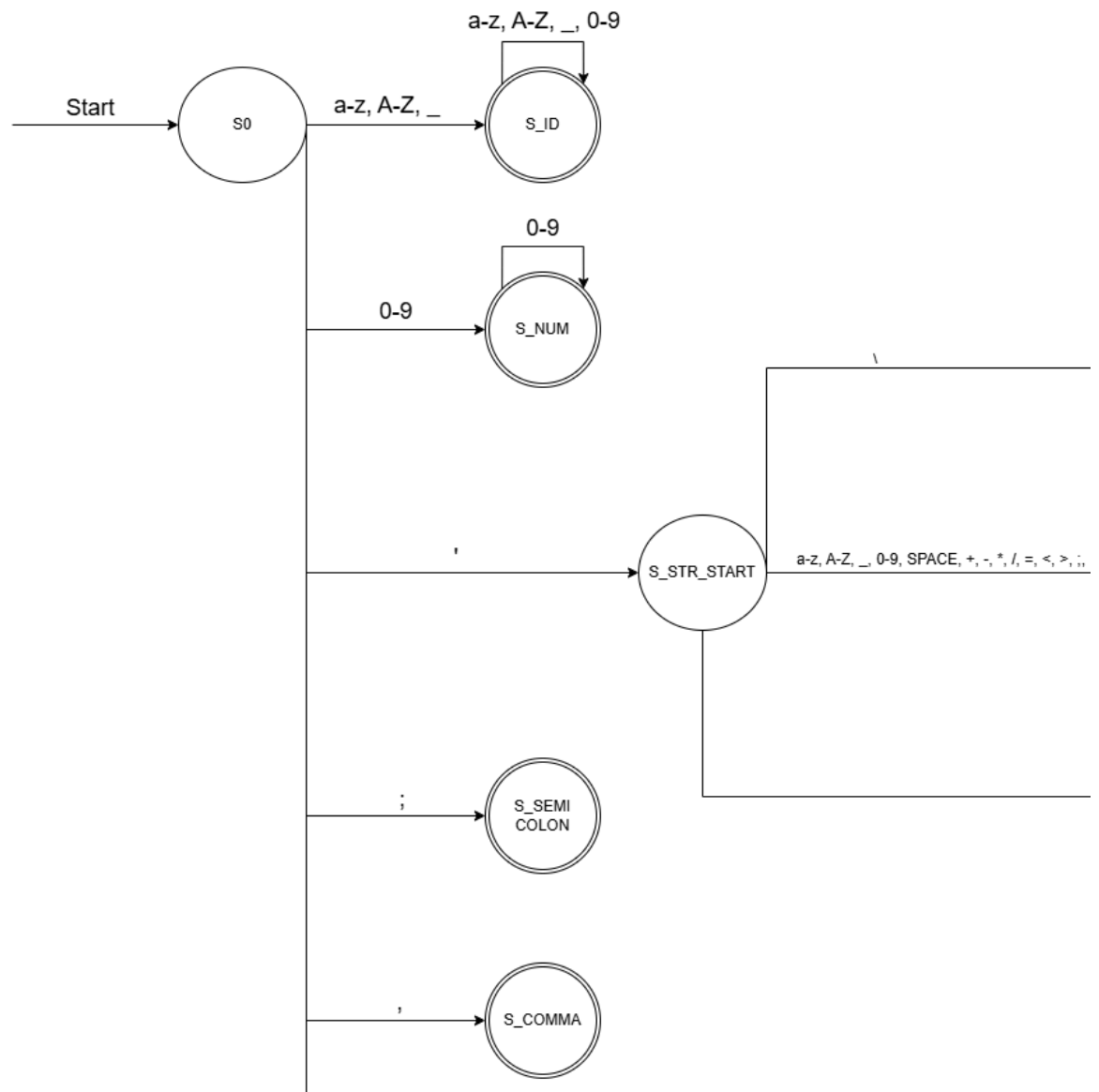
Dalam konteks analisis leksikal, DFA digunakan untuk mengenali pola karakter yang membentuk token. Langkah-langkahnya adalah sebagai berikut:

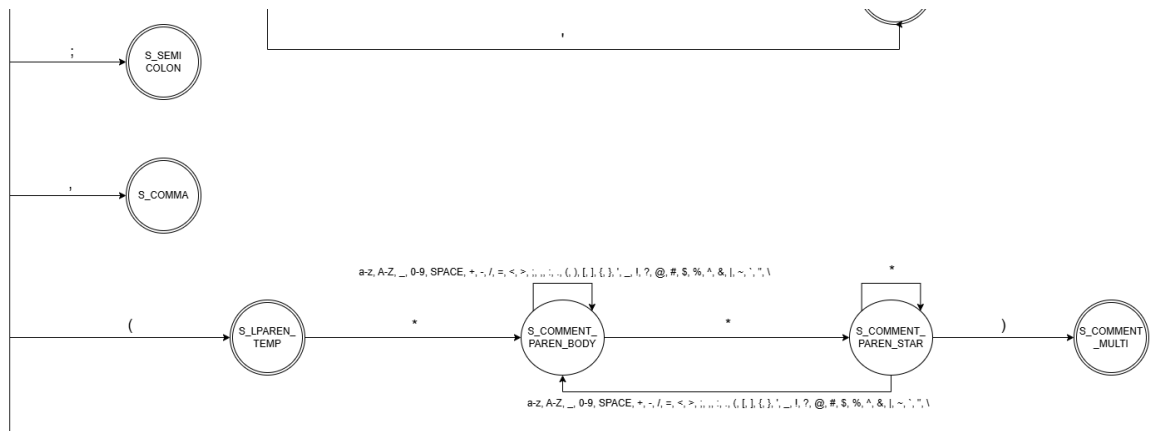
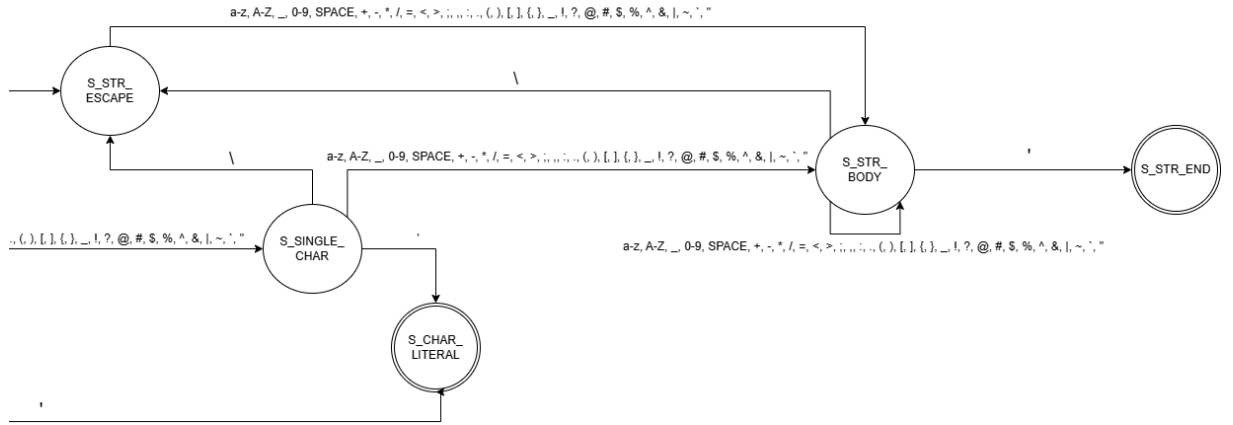
1. Inisialisasi: Lexer memulai dari keadaan awal DFA.
2. Pemrosesan Input: Untuk setiap karakter dalam input, tentukan keadaan berikutnya berdasarkan keadaan saat ini dan karakter input menggunakan fungsi transisi. Jika tidak ada transisi yang cocok, berarti kesalahan atau akhir token.
3. Pengenalan/Pengembalian Token: Ketika mencapai final state, lexer berarti sudah mengenali token yang valid.

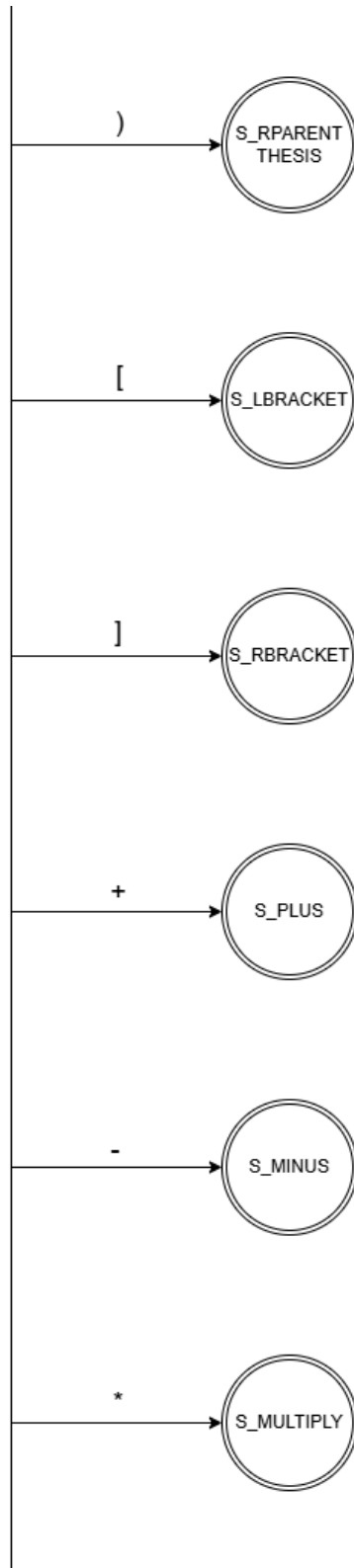
Suatu instans deterministic finite automata bergantung pada aturan-aturan yang dimilikinya. Dalam suatu implementasi, DFA mungkin akan membaca file, memproses konfigurasi, dan melakukan validasi apakah DFA valid atau tidak.

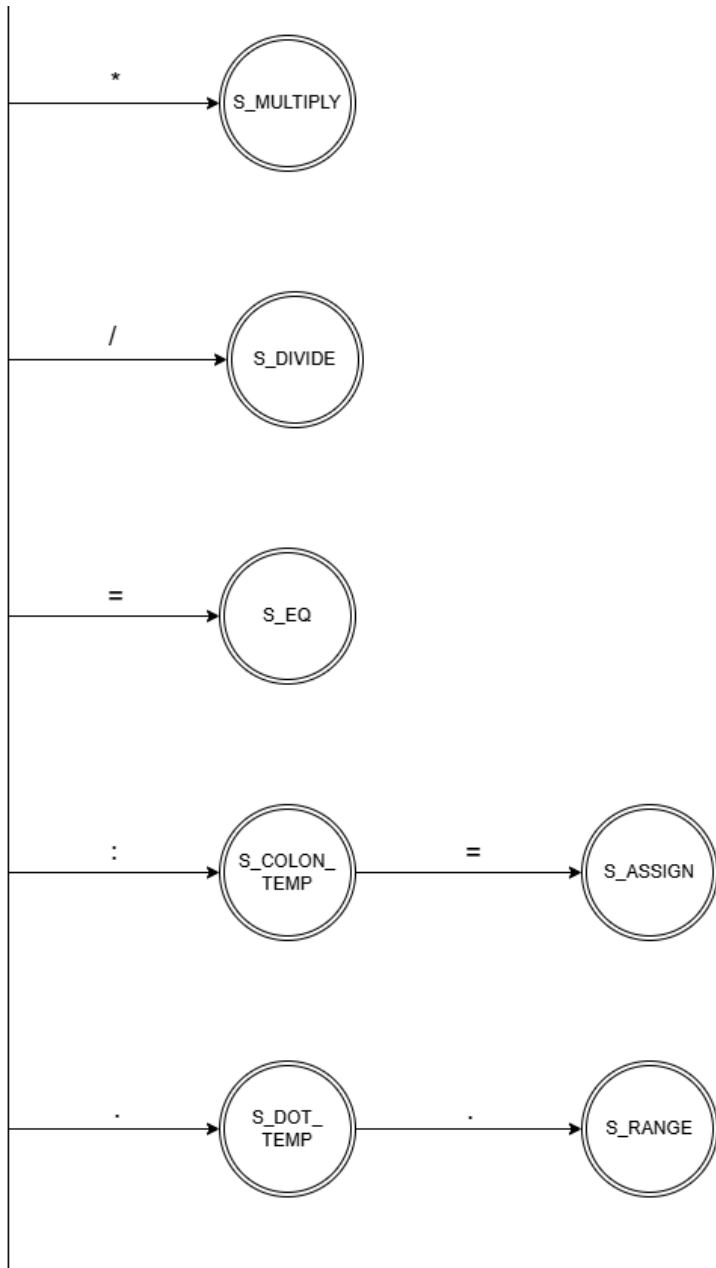
2. Perancangan & Implementasi

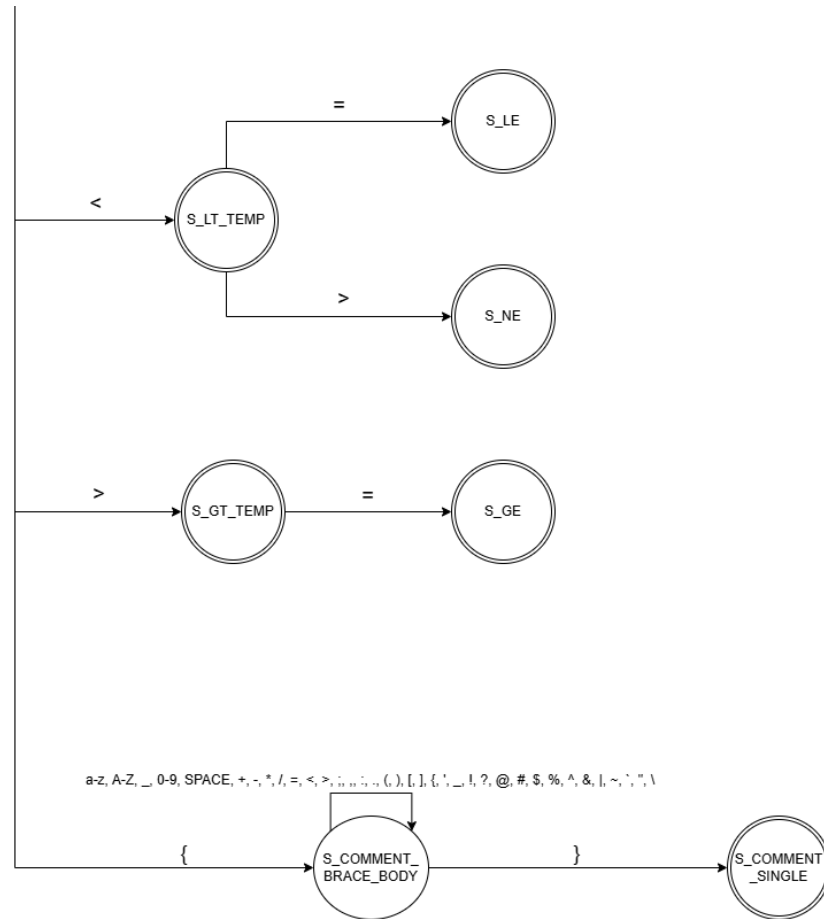
a. Diagram Deterministic Finite Automata (DFA)











Dari diagram DFA di atas, dapat diambil informasi sebagai berikut.

1. Start State : S0
2. Himpunan State : {S0, S_ID, S_NUM, S_CHAR_LITERAL, S_SEMICOLON, S_COMMA, S_COLON_TEMP, S_DOT_TEMP, S_LPAREN_TEMP, S_RPARENTHESIS, S_LBRACKET, S_RBRACKET, S_PLUS, S_MINUS, S_MULTIPLY, S_DIVIDE, S_ASSIGN, S_EQ, S_NE, S_LT_TEMP, S_LE, S_GT_TEMP, S_GE, S_RANGE, S_COMMENT_SINGLE, S_COMMENT_MULTI, S_STR_END, S_STR_START, S_STR_ESCAPE, S_STR_BODY, S_SINGLE_CHAR, S_COMMENT_PAREN_BODY, S_COMMENT_PAREN_STAR, S_COMMENT_BRACE_BODY}
3. Final State : {S_ID, S_NUM, S_CHAR_LITERAL, S_SEMICOLON, S_COMMA, S_COLON_TEMP, S_DOT_TEMP, S_LPAREN_TEMP, S_RPARENTHESIS, S_LBRACKET, S_RBRACKET, S_PLUS, S_MINUS, S_MULTIPLY, S_DIVIDE, S_ASSIGN, S_EQ, S_NE, S_LT_TEMP, S_LE, S_GT_TEMP, S_GE, S_RANGE, S_COMMENT_SINGLE, S_COMMENT_MULTI, S_STR_END}

Diagram DFA ini akan digunakan sebagai bahan untuk melakukan analisis leksikal. Dengan DFA ini, compiler akan membaca rangkaian karakter dari sumber program dan mengubahnya menjadi rangkaian satuan makna yang disebut token. Token merupakan unit terkecil yang memiliki arti dalam sebuah program, misalnya kata kunci (keyword), nama variabel (identifier), operator, angka, atau tanda baca. Berikut adalah contoh penggunaan DFA di atas untuk mengenali pola karakter atau token.

1. Dimulai dari start state (S0), DFA siap menerima input karakter.
2. Dari start state (S0), apabila DFA membaca karakter `;`, terjadi perubahan state yang awalnya S0 menjadi S_SEMICOLON.
3. S_SEMICOLON merupakan salah satu final state. Hal ini menunjukkan bahwa DFA berhasil membaca karakter dan diterima sebagai token SEMICOLON.

b. Implementasi Tambahan

Dalam proses perancangan DFA ini, dibuat implementasi tambahan yaitu state mapping yang berfungsi untuk melakukan pemetaan final state ke sebuah tipe token. Berikut adalah kode implementasinya.

```
// Initialize DFA state to token type mapping
void Lexer::initializeStateMapping() {
    stateToTokenType["S_ID"] = IDENTIFIER;
    stateToTokenType["S_NUM"] = NUMBER;

    stateToTokenType["S_CHAR_LITERAL"] = CHAR_LITERAL;
    stateToTokenType["S_SINGLE_CHAR"] = CHAR_LITERAL;

    stateToTokenType["S_STRING_LITERAL"] = STRING_LITERAL;
    stateToTokenType["S_STR_END"] = STRING_LITERAL;
    stateToTokenType["S_SEMICOLON"] = SEMICOLON;
    stateToTokenType["S_COMMA"] = COMMA;
    stateToTokenType["S_COLON_TEMP"] = COLON;
    stateToTokenType["S_DOT_TEMP"] = DOT;
    stateToTokenType["S_LPARENTHESIS"] = LPARENTHESIS;
    stateToTokenType["S_LPAREN_TEMP"] = LPARENTHESIS;
    stateToTokenType["S_RPARENTHESIS"] = RPARENTHESIS;
    stateToTokenType["S_LBRACKET"] = LBRACKET;
    stateToTokenType["S_RBRACKET"] = RBRACKET;
    stateToTokenType["S_PLUS"] = ARITHMETIC_OPERATOR;
    stateToTokenType["S_MINUS"] = ARITHMETIC_OPERATOR;
    stateToTokenType["S_MULTIPLY"] = ARITHMETIC_OPERATOR;
    stateToTokenType["S_DIVIDE"] = ARITHMETIC_OPERATOR;
    stateToTokenType["S_ASSIGN"] = ASSIGN_OPERATOR;
    stateToTokenType["S_EQ"] = RELATIONAL_OPERATOR;
    stateToTokenType["S_NE"] = RELATIONAL_OPERATOR;
    stateToTokenType["S_LT_TEMP"] = RELATIONAL_OPERATOR;
    stateToTokenType["S_LE"] = RELATIONAL_OPERATOR;
```

```

stateToTokenType["S_GT_TEMP"] = RELATIONAL_OPERATOR;
stateToTokenType["S_GE"] = RELATIONAL_OPERATOR;
stateToTokenType["S_RANGE"] = RANGE_OPERATOR;
}

```

3. Pengujian

a. base.pas

Input
<pre> program Hello; var a, b: integer; begin a := 5; b := a + 10; writeln(Result = , b); end. </pre>
Output
<pre> ----- KEYWORD(program) IDENTIFIER(Hello) SEMICOLON(;) KEYWORD(var) IDENTIFIER(a) COMMA(,) IDENTIFIER(b) COLON(:) KEYWORD(integer) SEMICOLON(;) KEYWORD(begin) IDENTIFIER(a) ASSIGN_OPERATOR(:=) NUMBER(5) SEMICOLON(;) IDENTIFIER(b) ASSIGN_OPERATOR(:=) IDENTIFIER(a) ARITHMETIC_OPERATOR(+) NUMBER(10) SEMICOLON(;) </pre>

```
IDENTIFIER(writeln)
LPARENTHESIS()
IDENTIFIER(Result)
RELATIONAL_OPERATOR(=)
COMMA(,)
IDENTIFIER(b)
RPARENTHESIS())
SEMICOLON(; )
KEYWORD(end)
DOT(.)
-----
Tokenization completed successfully!
Total tokens: 31
```

Bukti

```

raif@Raif:~/YRH-Tubes-IF2224$ ./bin/compiler ./test/milestone-1/base.pas
Using DFA-based lexer
DFA rules file: rules/pascal_lexicon.dfa
Processing file: ./test/milestone-1/base.pas
-----
KEYWORD(program)
IDENTIFIER>Hello)
SEMICOLON(;)
KEYWORD(var)
IDENTIFIER(a)
COMMA(,)
IDENTIFIER(b)
COLON(:)
KEYWORD(integer)
SEMICOLON(;)
KEYWORD(begin)
IDENTIFIER(a)
ASSIGN_OPERATOR(:=)
NUMBER(5)
SEMICOLON(;)
IDENTIFIER(b)
ASSIGN_OPERATOR(:=)
IDENTIFIER(a)
ARITHMETIC_OPERATOR(+)
NUMBER(10)
SEMICOLON(;)
IDENTIFIER>writeln)
LPARENTHESIS((
IDENTIFIER(Result)
RELATIONAL_OPERATOR(=)
COMMA(,)
IDENTIFIER(b)
RPARENTHESIS())
SEMICOLON(;)
KEYWORD(end)
DOT(.)
-----
Tokenization completed successfully!
Total tokens: 31

```

b. little_boy.pas

Input

```

program TestPascal;

const
    maxsize := 100;

```

```

var
  x, y, result: integer;
  flag: boolean;

procedure calculate;
begin
  x := 10;
  y := 20;
  result := x + y * 2;
  if result > 30 then
    flag := true
  else
    flag := false
end;

begin
  { This is a Pascal-S comment }
  call calculate;

  (* Another comment style *)
  while x < y do
  begin
    x := x + 1;
    if odd(x) then
      result := result - 1
    end;

    { String and character examples }
    writeln('Hello Pascal-S World!');
    writeln('A');

    { Relational operators }
    if x = y then result := 0;
    if x <> y then result := 1;
    if x <= y then result := 2;
    if x >= y then result := 3;

    { Range operator example }
    for i := 1..10 do
      result := result + i
    end.

```

Output

KEYWORD(program)

IDENTIFIER(TestPascal)
SEMICOLON(;)
KEYWORD(const)
IDENTIFIER(maxsize)
ASSIGN_OPERATOR(:=)
NUMBER(100)
SEMICOLON(;)
KEYWORD(var)
IDENTIFIER(x)
COMMA(,)
IDENTIFIER(y)
COMMA(,)
IDENTIFIER(result)
COLON(:)
KEYWORD(integer)
SEMICOLON(;)
IDENTIFIER(flag)
COLON(:)
KEYWORD(boolean)
SEMICOLON(;)
KEYWORD(procedure)
IDENTIFIER(calculate)
SEMICOLON(;)
KEYWORD(begin)
IDENTIFIER(x)
ASSIGN_OPERATOR(:=)
NUMBER(10)
SEMICOLON(;)
IDENTIFIER(y)
ASSIGN_OPERATOR(:=)
NUMBER(20)
SEMICOLON(;)
IDENTIFIER(result)
ASSIGN_OPERATOR(:=)
IDENTIFIER(x)
ARITHMETIC_OPERATOR(+)
IDENTIFIER(y)
ARITHMETIC_OPERATOR(*)
NUMBER(2)
SEMICOLON(;)
KEYWORD(if)
IDENTIFIER(result)
RELATIONAL_OPERATOR(>)
NUMBER(30)
KEYWORD(then)
IDENTIFIER(flag)

```
ASSIGN_OPERATOR(:=)
IDENTIFIER(true)
KEYWORD(else)
IDENTIFIER(flag)
ASSIGN_OPERATOR(:=)
IDENTIFIER(false)
KEYWORD(end)
SEMICOLON(;
KEYWORD(begin)
IDENTIFIER(call)
IDENTIFIER(calculate)
SEMICOLON(;
KEYWORD(while)
IDENTIFIER(x)
RELATIONAL_OPERATOR(<)
IDENTIFIER(y)
KEYWORD(do)
KEYWORD(begin)
IDENTIFIER(x)
ASSIGN_OPERATOR(:=)
IDENTIFIER(x)
ARITHMETIC_OPERATOR(+)
NUMBER(1)
SEMICOLON(;
KEYWORD(if)
IDENTIFIER(odd)
LPARENTHESIS()
IDENTIFIER(x)
RPARENTHESIS())
KEYWORD(then)
IDENTIFIER(result)
ASSIGN_OPERATOR(:=)
IDENTIFIER(result)
ARITHMETIC_OPERATOR(-)
NUMBER(1)
KEYWORD(end)
SEMICOLON(;
IDENTIFIER(writeln)
LPARENTHESIS()
STRING_LITERAL(Hello Pascal-S World!)
RPARENTHESIS())
SEMICOLON(;
IDENTIFIER(writeln)
LPARENTHESIS()
CHAR_LITERAL(A)
RPARENTHESIS())
```



```
SEMICOLON(;)
KEYWORD(if)
IDENTIFIER(x)
RELATIONAL_OPERATOR(=)
IDENTIFIER(y)
KEYWORD(then)
IDENTIFIER(result)
ASSIGN_OPERATOR(:=)
NUMBER(0)
SEMICOLON(;)
KEYWORD(if)
IDENTIFIER(x)
RELATIONAL_OPERATOR(<>)
IDENTIFIER(y)
KEYWORD(then)
IDENTIFIER(result)
ASSIGN_OPERATOR(:=)
NUMBER(1)
SEMICOLON(;)
KEYWORD(if)
IDENTIFIER(x)
RELATIONAL_OPERATOR(<=)
IDENTIFIER(y)
KEYWORD(then)
IDENTIFIER(result)
ASSIGN_OPERATOR(:=)
NUMBER(2)
SEMICOLON(;)
KEYWORD(if)
IDENTIFIER(x)
RELATIONAL_OPERATOR(>=)
IDENTIFIER(y)
KEYWORD(then)
IDENTIFIER(result)
ASSIGN_OPERATOR(:=)
NUMBER(3)
SEMICOLON(;)
KEYWORD(for)
IDENTIFIER(i)
ASSIGN_OPERATOR(:=)
NUMBER(1)
RANGE_OPERATOR(..)
NUMBER(10)
KEYWORD(do)
IDENTIFIER(result)
ASSIGN_OPERATOR(:=)
```

IDENTIFIER(result)
ARITHMETIC_OPERATOR(+)
IDENTIFIER(i)
KEYWORD(end)
DOT(.)

Tokenization completed successfully!
Total tokens: 144

Bukti

```
raif@Raif:~/YRH-Tubes-IF2224$ ./bin/compiler ./test/milestone-1/little_boy.pas
Using DFA-based lexer
DFA rules file: rules/pascal_lexicon.dfa
Processing file: ./test/milestone-1/little_boy.pas
-----
KEYWORD(program)
IDENTIFIER(TestPascal)
SEMICOLON(;)
KEYWORD(const)
IDENTIFIER(maxsize)
ASSIGN_OPERATOR(:=)
NUMBER(100)
SEMICOLON(;)
KEYWORD(var)
IDENTIFIER(x)
COMMA(,)
IDENTIFIER(y)
COMMA(,)
IDENTIFIER(result)
COLON(:)
KEYWORD(integer)
SEMICOLON(;
```

```

IDENTIFIER(result)
ASSIGN_OPERATOR(:=)
NUMBER(2)
SEMICOLON(;)
KEYWORD(if)
IDENTIFIER(x)
RELATIONAL_OPERATOR(>=)
IDENTIFIER(y)
KEYWORD(then)
IDENTIFIER(result)
ASSIGN_OPERATOR(:=)
NUMBER(3)
SEMICOLON(;)
KEYWORD(for)
IDENTIFIER(i)
ASSIGN_OPERATOR(:=)
NUMBER(1)
RANGE_OPERATOR(..)
NUMBER(10)
KEYWORD(do)
IDENTIFIER(result)
ASSIGN_OPERATOR(:=)
IDENTIFIER(result)
ARITHMETIC_OPERATOR(+)
IDENTIFIER(i)
KEYWORD(end)
DOT(.)
-----
Tokenization completed successfully!
Total tokens: 144

```

c. **big_guy.pas**

Input

```

program ComprehensiveTest;

const
  MAX_SIZE = 100;
  MIN_VALUE = 0;
  DEFAULT_CHAR = 'X';

var
  counter, total, result: integer;
  flag, done: boolean;
  current_char: char;
  message, name: string;

procedure initialize;

```

```

begin
    { Initialize variables with different literal types }
    counter := 0;
    total := 1000;
    flag := true;
    done := false;
    current_char := 'A';
    message := 'Starting initialization';
    name := 'Pascal-S Program'
end;

procedure calculate;
var
    temp: integer;
begin
    { Arithmetic operations }
    temp := counter + total;
    temp := temp - MIN_VALUE;
    temp := temp * 2;
    temp := temp / 4;

    { Assignment operations }
    result := temp;
    counter := counter + 1
end;

procedure testRelationalOperators;
begin
    { All relational operators }
    if counter = 10 then result := 1;
    if counter <> 10 then result := 2;
    if counter < MAX_SIZE then result := 3;
    if counter <= MAX_SIZE then result := 4;
    if counter > MIN_VALUE then result := 5;
    if counter >= MIN_VALUE then result := 6
end;

procedure testControlFlow;
begin
    { While loop with relational operators }
    while counter < 10 do
    begin
        counter := counter + 1;
        if odd(counter) then
            result := result + counter
        else

```

```

        result := result - 1
    end;

    { Conditional statements }
    if result > 50 then
    begin
        flag := true;
        message := 'Result is high'
    end
    else
    begin
        flag := false;
        message := 'Result is low'
    end
end;

procedure testCharactersAndStrings;
begin
    { Character literals - single characters }
    current_char := 'Z';
    current_char := '9';
    current_char := ' ';
    current_char := '\n';
    current_char := '\t';
    current_char := "\";
    current_char := '\\';

    { String literals - multiple characters }
    message := 'Hello Pascal-S World!';
    message := 'String with numbers: 12345';
    message := 'String with special chars: @#$$%^&*()';
    message := ""; { Empty string }
    message := 'Escaped quotes: \'Hello\' and \'World\'';
    message := 'Multi-line\nstring with\ttabs';

    { Mixed usage }
    if current_char = 'A' then
        message := 'Character is A'
    else if current_char = 'Z' then
        message := 'Character is Z'
    else
        message := 'Character is something else'
    end;

procedure testRangeOperator;
var

```

```

i: integer;
begin
  { Range operator in for loops }
  for i := 1..10 do
    total := total + i;

    for i := 0..MAX_SIZE do
      if i = counter then
        done := true
      end if;
    end for;
  end for;
end;

begin
  (* Main program starts here *)
  { This is a brace-style comment }

  { Call all test procedures }
  call initialize;
  call calculate;
  call testRelationalOperators;
  call testControlFlow;
  call testCharactersAndStrings;
  call testRangeOperator;

  { Final operations }
  if done then
    message := 'All tests completed successfully'
  else
    message := 'Some tests may have failed';
  end if;

  { Demonstrate all punctuation }
  result := (counter + total) * 2;

  { Program termination }
  done := true
end.

```

Output

```

KEYWORD(program)
IDENTIFIER(ComprehensiveTest)
SEMICOLON(;)
KEYWORD(const)
IDENTIFIER(MAX_SIZE)
RELATIONAL_OPERATOR(=)

```

```
NUMBER(100)
SEMICOLON(;)
IDENTIFIER(MIN_VALUE)
RELATIONAL_OPERATOR(=)
NUMBER(0)
SEMICOLON(;)
IDENTIFIER(DEFAULT_CHAR)
RELATIONAL_OPERATOR(=)
CHAR_LITERAL(X)
SEMICOLON(;)
KEYWORD(var)
IDENTIFIER(counter)
COMMA(,)
IDENTIFIER(total)
COMMA(,)
IDENTIFIER(result)
COLON(:)
KEYWORD(integer)
SEMICOLON(;)
IDENTIFIER(flag)
COMMA(,)
IDENTIFIER(done)
COLON(:)
KEYWORD(boolean)
SEMICOLON(;)
IDENTIFIER(current_char)
COLON(:)
KEYWORD(char)
SEMICOLON(;)
IDENTIFIER(message)
COMMA(,)
IDENTIFIER(name)
COLON(:)
IDENTIFIER(string)
SEMICOLON(;)
KEYWORD(procedure)
IDENTIFIER(initialize)
SEMICOLON(;)
KEYWORD(begin)
IDENTIFIER(counter)
ASSIGN_OPERATOR(:=)
NUMBER(0)
SEMICOLON(;)
IDENTIFIER(total)
ASSIGN_OPERATOR(:=)
NUMBER(1000)
```

```
SEMICOLON(;)
IDENTIFIER(flag)
ASSIGN_OPERATOR(:=)
IDENTIFIER(true)
SEMICOLON(;)
IDENTIFIER(done)
ASSIGN_OPERATOR(:=)
IDENTIFIER(false)
SEMICOLON(;)
IDENTIFIER(current_char)
ASSIGN_OPERATOR(:=)
CHAR_LITERAL(A)
SEMICOLON(;)
IDENTIFIER(message)
ASSIGN_OPERATOR(:=)
STRING_LITERAL(Starting initialization)
SEMICOLON(;)
IDENTIFIER(name)
ASSIGN_OPERATOR(:=)
STRING_LITERAL(Pascal-S Program)
KEYWORD(end)
SEMICOLON(;)
KEYWORD(procedure)
IDENTIFIER(calculate)
SEMICOLON(;)
KEYWORD(var)
IDENTIFIER(temp)
COLON(:)
KEYWORD(integer)
SEMICOLON(;)
KEYWORD(begin)
IDENTIFIER(temp)
ASSIGN_OPERATOR(:=)
IDENTIFIER(counter)
ARITHMETIC_OPERATOR(+)
IDENTIFIER(total)
SEMICOLON(;)
IDENTIFIER(temp)
ASSIGN_OPERATOR(:=)
IDENTIFIER(temp)
ARITHMETIC_OPERATOR(-)
IDENTIFIER(MIN_VALUE)
SEMICOLON(;)
IDENTIFIER(temp)
ASSIGN_OPERATOR(:=)
IDENTIFIER(temp)
```



```
ARITHMETIC_OPERATOR(*)
NUMBER(2)
SEMICOLON(;);
IDENTIFIER(temp)
ASSIGN_OPERATOR(:=)
IDENTIFIER(temp)
ARITHMETIC_OPERATOR(/)
NUMBER(4)
SEMICOLON(;);
IDENTIFIER(result)
ASSIGN_OPERATOR(:=)
IDENTIFIER(temp)
SEMICOLON(;);
IDENTIFIER(counter)
ASSIGN_OPERATOR(:=)
IDENTIFIER(counter)
ARITHMETIC_OPERATOR(+)
NUMBER(1)
KEYWORD(end)
SEMICOLON(;);
KEYWORD(procedure)
IDENTIFIER(testRelationalOperators)
SEMICOLON(;);
KEYWORD(begin)
KEYWORD(if)
IDENTIFIER(counter)
RELATIONAL_OPERATOR(=)
NUMBER(10)
KEYWORD(then)
IDENTIFIER(result)
ASSIGN_OPERATOR(:=)
NUMBER(1)
SEMICOLON(;);
KEYWORD(if)
IDENTIFIER(counter)
RELATIONAL_OPERATOR(<>)
NUMBER(10)
KEYWORD(then)
IDENTIFIER(result)
ASSIGN_OPERATOR(:=)
NUMBER(2)
SEMICOLON(;);
KEYWORD(if)
IDENTIFIER(counter)
RELATIONAL_OPERATOR(<)
IDENTIFIER(MAX_SIZE)
```

KEYWORD(then)
IDENTIFIER(result)
ASSIGN_OPERATOR(:=)
NUMBER(3)
SEMICOLON(;)
KEYWORD(if)
IDENTIFIER(counter)
RELATIONAL_OPERATOR(<=)
IDENTIFIER(MAX_SIZE)
KEYWORD(then)
IDENTIFIER(result)
ASSIGN_OPERATOR(:=)
NUMBER(4)
SEMICOLON(;)
KEYWORD(if)
IDENTIFIER(counter)
RELATIONAL_OPERATOR(>)
IDENTIFIER(MIN_VALUE)
KEYWORD(then)
IDENTIFIER(result)
ASSIGN_OPERATOR(:=)
NUMBER(5)
SEMICOLON(;)
KEYWORD(if)
IDENTIFIER(counter)
RELATIONAL_OPERATOR(>=)
IDENTIFIER(MIN_VALUE)
KEYWORD(then)
IDENTIFIER(result)
ASSIGN_OPERATOR(:=)
NUMBER(6)
KEYWORD(end)
SEMICOLON(;)
KEYWORD(procedure)
IDENTIFIER(testControlFlow)
SEMICOLON(;)
KEYWORD(begin)
KEYWORD(while)
IDENTIFIER(counter)
RELATIONAL_OPERATOR(<)
NUMBER(10)
KEYWORD(do)
KEYWORD(begin)
IDENTIFIER(counter)
ASSIGN_OPERATOR(:=)
IDENTIFIER(counter)

```
ARITHMETIC_OPERATOR(+)  
NUMBER(1)  
SEMICOLON(;)  
KEYWORD(if)  
IDENTIFIER(odd)  
LPARENTHESIS(  
IDENTIFIER(counter)  
RPARENTHESIS())  
KEYWORD(then)  
IDENTIFIER(result)  
ASSIGN_OPERATOR(:=)  
IDENTIFIER(result)  
ARITHMETIC_OPERATOR(+)  
IDENTIFIER(counter)  
KEYWORD(else)  
IDENTIFIER(result)  
ASSIGN_OPERATOR(:=)  
IDENTIFIER(result)  
ARITHMETIC_OPERATOR(-)  
NUMBER(1)  
KEYWORD(end)  
SEMICOLON(;)  
KEYWORD(if)  
IDENTIFIER(result)  
RELATIONAL_OPERATOR(>)  
NUMBER(50)  
KEYWORD(then)  
KEYWORD(begin)  
IDENTIFIER(flag)  
ASSIGN_OPERATOR(:=)  
IDENTIFIER(true)  
SEMICOLON(;)  
IDENTIFIER(message)  
ASSIGN_OPERATOR(:=)  
STRING_LITERAL(Result is high)  
KEYWORD(end)  
KEYWORD(else)  
KEYWORD(begin)  
IDENTIFIER(flag)  
ASSIGN_OPERATOR(:=)  
IDENTIFIER(false)  
SEMICOLON(;)  
IDENTIFIER(message)  
ASSIGN_OPERATOR(:=)  
STRING_LITERAL(Result is low)  
KEYWORD(end)
```

```
KEYWORD(end)
SEMICOLON(;)
KEYWORD(procedure)
IDENTIFIER(testCharactersAndStrings)
SEMICOLON(;)
KEYWORD(begin)
IDENTIFIER(current_char)
ASSIGN_OPERATOR(:=)
CHAR_LITERAL(Z)
SEMICOLON(;)
IDENTIFIER(current_char)
ASSIGN_OPERATOR(:=)
CHAR_LITERAL(9)
SEMICOLON(;)
IDENTIFIER(current_char)
ASSIGN_OPERATOR(:=)
CHAR_LITERAL( )
SEMICOLON(;)
IDENTIFIER(current_char)
ASSIGN_OPERATOR(:=)
CHAR_LITERAL(
)
SEMICOLON(;)
IDENTIFIER(current_char)
ASSIGN_OPERATOR(:=)
CHAR_LITERAL( )
SEMICOLON(;)
IDENTIFIER(current_char)
ASSIGN_OPERATOR(:=)
CHAR_LITERAL(')
SEMICOLON(;)
IDENTIFIER(current_char)
ASSIGN_OPERATOR(:=)
CHAR_LITERAL(\)
SEMICOLON(;)
IDENTIFIER(message)
ASSIGN_OPERATOR(:=)
STRING_LITERAL(Hello Pascal-S World!)
SEMICOLON(;)
IDENTIFIER(message)
ASSIGN_OPERATOR(:=)
STRING_LITERAL(String with numbers: 12345)
SEMICOLON(;)
IDENTIFIER(message)
ASSIGN_OPERATOR(:=)
STRING_LITERAL(String with special chars: @#$$%^&*())
```

```
SEMICOLON(;)
IDENTIFIER(message)
ASSIGN_OPERATOR(:=)
CHAR_LITERAL()
SEMICOLON(;)
IDENTIFIER(message)
ASSIGN_OPERATOR(:=)
STRING_LITERAL(Escaped quotes: 'Hello' and 'World')
SEMICOLON(;)
IDENTIFIER(message)
ASSIGN_OPERATOR(:=)
STRING_LITERAL(Multi-line
string with  tabs)
SEMICOLON(;)
KEYWORD(if)
IDENTIFIER(current_char)
RELATIONAL_OPERATOR(=)
CHAR_LITERAL(A)
KEYWORD(then)
IDENTIFIER(message)
ASSIGN_OPERATOR(:=)
STRING_LITERAL(Character is A)
KEYWORD(else)
KEYWORD(if)
IDENTIFIER(current_char)
RELATIONAL_OPERATOR(=)
CHAR_LITERAL(Z)
KEYWORD(then)
IDENTIFIER(message)
ASSIGN_OPERATOR(:=)
STRING_LITERAL(Character is Z)
KEYWORD(else)
IDENTIFIER(message)
ASSIGN_OPERATOR(:=)
STRING_LITERAL(Character is something else)
KEYWORD(end)
SEMICOLON(;)
KEYWORD(procedure)
IDENTIFIER(testRangeOperator)
SEMICOLON(;)
KEYWORD(var)
IDENTIFIER(i)
COLON(:)
KEYWORD(integer)
SEMICOLON(;)
KEYWORD(begin)
```

```
KEYWORD(for)
IDENTIFIER(i)
ASSIGN_OPERATOR(:=)
NUMBER(1)
RANGE_OPERATOR(..)
NUMBER(10)
KEYWORD(do)
IDENTIFIER(total)
ASSIGN_OPERATOR(:=)
IDENTIFIER(total)
ARITHMETIC_OPERATOR(+)
IDENTIFIER(i)
SEMICOLON(; )
KEYWORD(for)
IDENTIFIER(i)
ASSIGN_OPERATOR(:=)
NUMBER(0)
RANGE_OPERATOR(..)
IDENTIFIER(MAX_SIZE)
KEYWORD(do)
KEYWORD(if)
IDENTIFIER(i)
RELATIONAL_OPERATOR(=)
IDENTIFIER(counter)
KEYWORD(then)
IDENTIFIER(done)
ASSIGN_OPERATOR(:=)
IDENTIFIER(true)
KEYWORD(end)
SEMICOLON(; )
KEYWORD(begin)
IDENTIFIER(call)
IDENTIFIER(initialize)
SEMICOLON(; )
IDENTIFIER(call)
IDENTIFIER(calculate)
SEMICOLON(; )
IDENTIFIER(call)
IDENTIFIER(testRelationalOperators)
SEMICOLON(; )
IDENTIFIER(call)
IDENTIFIER(testControlFlow)
SEMICOLON(; )
IDENTIFIER(call)
IDENTIFIER(testCharactersAndStrings)
SEMICOLON(; )
```

```
IDENTIFIER(call)
IDENTIFIER(testRangeOperator)
SEMICOLON(;)
KEYWORD(if)
IDENTIFIER(done)
KEYWORD(then)
IDENTIFIER(message)
ASSIGN_OPERATOR(:=)
STRING_LITERAL(All tests completed successfully)
KEYWORD(else)
IDENTIFIER(message)
ASSIGN_OPERATOR(:=)
STRING_LITERAL(Some tests may have failed)
SEMICOLON(;)
IDENTIFIER(result)
ASSIGN_OPERATOR(:=)
LPARENTHESIS(
IDENTIFIER(counter)
ARITHMETIC_OPERATOR(+)
IDENTIFIER(total)
RPARENTHESIS())
ARITHMETIC_OPERATOR(*)
NUMBER(2)
SEMICOLON(;)
IDENTIFIER(done)
ASSIGN_OPERATOR(:=)
IDENTIFIER(true)
KEYWORD(end)
DOT(.)
```

Tokenization completed successfully!
Total tokens: 401

Bukti

```

raif@Raif:~/YRH-Tubes-IF2224$ ./bin/compiler ./test/milestone-1/big_guy.pas
Using DFA-based lexer
DFA rules file: rules/pascal_lexicon.dfa
Processing file: ./test/milestone-1/big_guy.pas

```

```

-----
KEYWORD(program)
IDENTIFIER(ComprehensiveTest)
SEMICOLON(;)
KEYWORD(const)
IDENTIFIER(MAX_SIZE)
RELATIONAL_OPERATOR(=)
NUMBER(100)
SEMICOLON(;)
IDENTIFIER(MIN_VALUE)
RELATIONAL_OPERATOR(=)
NUMBER(0)
SEMICOLON(;)

```

```

STRING_LITERAL(All tests completed successfully)
KEYWORD(else)
IDENTIFIER(message)
ASSIGN_OPERATOR(:=)
STRING_LITERAL(Some tests may have failed)
SEMICOLON(;)
IDENTIFIER(result)
ASSIGN_OPERATOR(:=)
LPARENTHESIS((
IDENTIFIER(counter)
ARITHMETIC_OPERATOR(+)
IDENTIFIER(total)
RPARENTHESIS())
ARITHMETIC_OPERATOR(*)
NUMBER(2)
SEMICOLON(;)
IDENTIFIER(done)
ASSIGN_OPERATOR(:=)
IDENTIFIER(true)
KEYWORD(end)
DOT(.)
-----
Tokenization completed successfully!
Total tokens: 401

```

d. medium_girl.pas

Input

```

program LiteralTest;

const
  PI = 3.14;
  GREETING = 'Hello, Lexer!';
  EMPTY = "";

```



```

var
  ch: char;
  str: string;
  num: integer;

begin
  ch := 'X';
  str := GREETING;
  num := 42;
  writeln('Char:', ch);
  writeln('String:', str);
  writeln('Number:', num);
  writeln('Empty string:', EMPTY);
  writeln('Pi:', PI)
end.

```

Output

```

KEYWORD(program)
IDENTIFIER(LiteralTest)
SEMICOLON(;)
KEYWORD(const)
IDENTIFIER(PI)
RELATIONAL_OPERATOR(=)
NUMBER(3)
DOT(.)
NUMBER(14)
SEMICOLON(;)
IDENTIFIER(GREETING)
RELATIONAL_OPERATOR(=)
STRING_LITERAL(Hello, Lexer!)
SEMICOLON(;)
IDENTIFIER(EMPTY)
RELATIONAL_OPERATOR(=)
CHAR_LITERAL()
SEMICOLON(;)
KEYWORD(var)
IDENTIFIER(ch)
COLON(:)
KEYWORD(char)
SEMICOLON(;)
IDENTIFIER(str)
COLON(:)
IDENTIFIER(string)
SEMICOLON(;)

```

```
IDENTIFIER(num)
COLON(:)
KEYWORD(integer)
SEMICOLON(;
KEYWORD(begin)
IDENTIFIER(ch)
ASSIGN_OPERATOR(:=)
CHAR_LITERAL(X)
SEMICOLON(;
IDENTIFIER(str)
ASSIGN_OPERATOR(:=)
IDENTIFIER(GREETING)
SEMICOLON(;
IDENTIFIER(num)
ASSIGN_OPERATOR(:=)
NUMBER(42)
SEMICOLON(;
IDENTIFIER(writeln)
LPARENTHESIS()
STRING_LITERAL(Char:)
COMMA(,)
IDENTIFIER(ch)
RPARENTHESIS())
SEMICOLON(;
IDENTIFIER(writeln)
LPARENTHESIS()
STRING_LITERAL(String:)
COMMA(,)
IDENTIFIER(str)
RPARENTHESIS())
SEMICOLON(;
IDENTIFIER(writeln)
LPARENTHESIS()
STRING_LITERAL(Number:)
COMMA(,)
IDENTIFIER(num)
RPARENTHESIS())
SEMICOLON(;
IDENTIFIER(writeln)
LPARENTHESIS()
STRING_LITERAL(Empty string:)
COMMA(,)
IDENTIFIER(EMPTY)
RPARENTHESIS())
SEMICOLON(;
IDENTIFIER(writeln)
```

LPARENTHESIS()
STRING_LITERAL(Pi:)
COMMA(,)
IDENTIFIER(PI)
RPARENTHESIS()
KEYWORD(end)
DOT(.)

Tokenization completed successfully!
Total tokens: 80

Bukti

```
raif@Raif:~/YRH-Tubes-IF2224$ ./bin/compiler ./test/milestone-1/medium_girl.pas
Using DFA-based lexer
DFA rules file: rules/pascal_lexicon.dfa
Processing file: ./test/milestone-1/medium_girl.pas
-----
KEYWORD(program)
IDENTIFIER(LiteralTest)
SEMICOLON(;)
KEYWORD(const)
IDENTIFIER(PI)
RELATIONAL_OPERATOR(=)
NUMBER(3)
DOT(.)
NUMBER(14)
SEMICOLON(;)
IDENTIFIER(GREETING)
RELATIONAL_OPERATOR(=)
STRING_LITERAL>Hello, Lexer!
SEMICOLON(;)
IDENTIFIER(EMPTY)
```

```

RPARENTHESIS())
SEMICOLON(;)
IDENTIFIER(writeln)
LPARENTHESIS()
STRING_LITERAL(Empty string:)
COMMA(,)
IDENTIFIER(EMPTY)
RPARENTHESIS())
SEMICOLON(;)
IDENTIFIER(writeln)
LPARENTHESIS()
STRING_LITERAL(Pi:)
COMMA(,)
IDENTIFIER(PI)
RPARENTHESIS())
KEYWORD(end)
DOT(.)
-----
Tokenization completed successfully!
Total tokens: 80

```

e. **small_lad.pas**

Input

```

program ControlFlowTest;

var
  i, sum: integer;
  status: boolean;

begin
  sum := 0;
  status := false;
  for i := 1 to 5 do
  begin
    sum := sum + i;
    if sum mod 2 = 0 then
      status := true
    else
      status := false
  end;
  while sum > 0 do
  begin

```

```
    sum := sum - 1
end;
writeln('Final sum:', sum);
writeln('Status:', status)
end.
```

Output

KEYWORD(program)
IDENTIFIER(ControlFlowTest)
SEMICOLON(;)
KEYWORD(var)
IDENTIFIER(i)
COMMA(,)
IDENTIFIER(sum)
COLON(:)
KEYWORD(integer)
SEMICOLON(;)
IDENTIFIER(status)
COLON(:)
KEYWORD(boolean)
SEMICOLON(;)
KEYWORD(begin)
IDENTIFIER(sum)
ASSIGN_OPERATOR(:=)
NUMBER(0)
SEMICOLON(;)
IDENTIFIER(status)
ASSIGN_OPERATOR(:=)
IDENTIFIER(false)
SEMICOLON(;)
KEYWORD(for)
IDENTIFIER(i)
ASSIGN_OPERATOR(:=)
NUMBER(1)
KEYWORD(to)
NUMBER(5)
KEYWORD(do)
KEYWORD(begin)
IDENTIFIER(sum)
ASSIGN_OPERATOR(:=)
IDENTIFIER(sum)
ARITHMETIC_OPERATOR(+)
IDENTIFIER(i)
SEMICOLON(;)
KEYWORD(if)

```
IDENTIFIER(sum)
ARITHMETIC_OPERATOR(mod)
NUMBER(2)
RELATIONAL_OPERATOR(=)
NUMBER(0)
KEYWORD(then)
IDENTIFIER(status)
ASSIGN_OPERATOR(:=)
IDENTIFIER(true)
KEYWORD(else)
IDENTIFIER(status)
ASSIGN_OPERATOR(:=)
IDENTIFIER(false)
KEYWORD(end)
SEMICOLON(;)
KEYWORD(while)
IDENTIFIER(sum)
RELATIONAL_OPERATOR(>)
NUMBER(0)
KEYWORD(do)
KEYWORD(begin)
IDENTIFIER(sum)
ASSIGN_OPERATOR(:=)
IDENTIFIER(sum)
ARITHMETIC_OPERATOR(-)
NUMBER(1)
KEYWORD(end)
SEMICOLON(;)
IDENTIFIER(writeln)
LPARENTHESIS()
STRING_LITERAL(Final sum:)
COMMA(,)
IDENTIFIER(sum)
RPARENTHESIS())
SEMICOLON(;)
IDENTIFIER(writeln)
LPARENTHESIS()
STRING_LITERAL(Status:)
COMMA(,)
IDENTIFIER(status)
RPARENTHESIS())
KEYWORD(end)
DOT(.)
```

Tokenization completed successfully!
Total tokens: 81

Bukti

```
raif@Raif:~/YRH-Tubes-IF2224$ ./bin/compiler ./test/milestone-1/small_lad.pas
Using DFA-based lexer
DFA rules file: rules/pascal_lexicon.dfa
Processing file: ./test/milestone-1/small_lad.pas
-----
KEYWORD(program)
IDENTIFIER(ControlFlowTest)
SEMICOLON(; )
KEYWORD(var)
IDENTIFIER(i)
COMMA(, )
IDENTIFIER(sum)
COLON(:)
KEYWORD(integer)
SEMICOLON(; )
IDENTIFIER(status)
COLON(:)

SEMICOLON(; )
IDENTIFIER(writeln)
LPARENTHESIS(( )
STRING_LITERAL(Status:)
COMMA(, )
IDENTIFIER(status)
RPARENTHESIS( )
KEYWORD(end)
DOT(.)
-----
Tokenization completed successfully!
Total tokens: 81
```

4. Kesimpulan & Saran

a. Kesimpulan

Implementasi *lexical analyzer* untuk Milestone 1 Tugas Besar IF3170 telah berhasil diselesaikan dengan menerapkan konsep Deterministic Finite Automaton (DFA). Lexer yang dibangun mampu menjalankan fungsinya, yaitu membaca rangkaian karakter dari *source code* Pascal-S dan mengubahnya menjadi serangkaian token yang valid.

Berdasarkan hasil pengujian terhadap lima file uji (base.pas, little_boy.pas, big_guy.pas, medium_girl.pas, dan small_lad.pas), lexer telah mampu untuk:

1. Mengenali semua token yang valid, termasuk KEYWORD, IDENTIFIER, NUMBER, STRING_LITERAL, dan CHAR_LITERAL, token-token operator seperti ASSIGN_OPERATOR, ARITHMETIC_OPERATOR, dan

RELATIONAL_OPERATOR, serta token-token simbol seperti titik, koma, titik koma, kurung, dsb.

2. Mengenali dan mengabaikan elemen-elemen yang tidak relevan untuk analisis syntax seperti whitespace dan komentar.

Secara keseluruhan, lexer yang diimplementasikan telah fungsional dan memenuhi spesifikasi yang dibutuhkan untuk milestone ini. Lexer ini dapat digunakan lebih lanjut ke tahap Milestone

b. Saran

Seluruh kasus uji yang disertakan berfokus pada *source code* yang valid secara leksikal. Lexer sebaiknya dikembangkan agar lebih kuat dengan menambahkan mekanisme *error handling*. Ini mencakup kemampuan mendeteksi dan melaporkan input yang tidak valid, seperti string literal atau komentar yang tidak ditutup, atau penggunaan simbol ilegal yang tidak terdefinisi dalam bahasa.

Lampiran

- Link Repository Github:
[Darsua/YRH-Tubes-IF2224: NieR: Finite Automata](#)
- Link Workspace Diagram:
<https://drive.google.com/file/d/1HY1pBAA-bTESXUx-YMbmtS0i8zcpjqBb/view?usp=sharing>
- Pembagian Tugas

Nama - NIM	Tugas	Kontribusi
Muhammad Ra'if Alkautsar - 13523011	Diagram DFA	40%
	Laporan Milestone 1	50%
Fajar Kurniawan - 13523027	Rules DFA	80%
	Laporan Milestone 1	20%
Darrel Adinarya Sunanda - 13523061	Implementasi Lexical Analyzer	100%
Reza Ahmad Syarif - 13523119	Rules DFA	20%
	Diagram DFA	60%
	Laporan Milestone 1	30%